

Threads

Threads (Motivation)



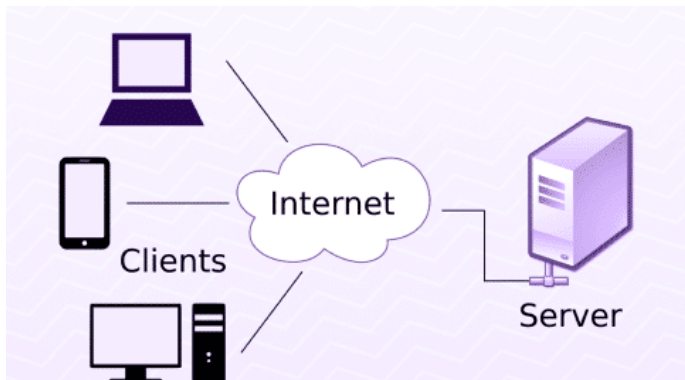
Games



Word Processing



Games



Multithreaded Server Architecture

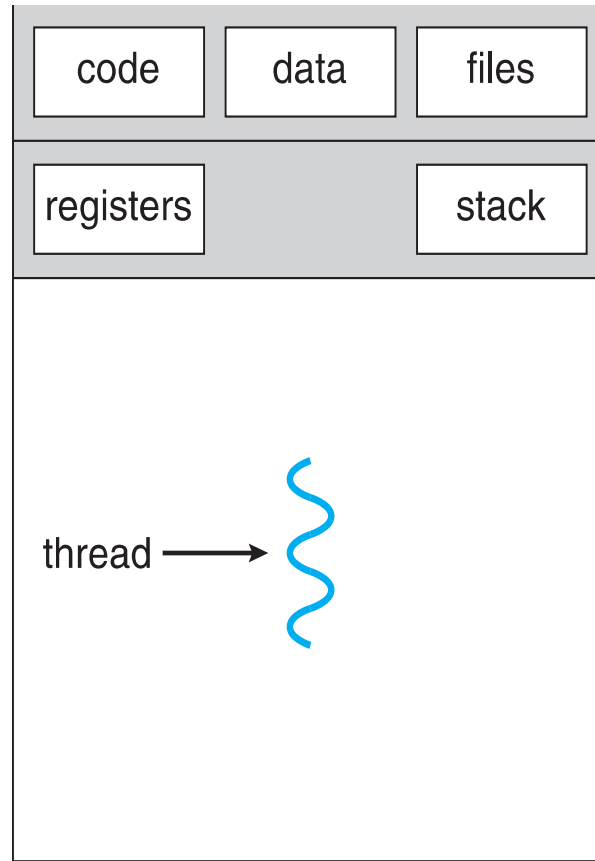


Browser Page

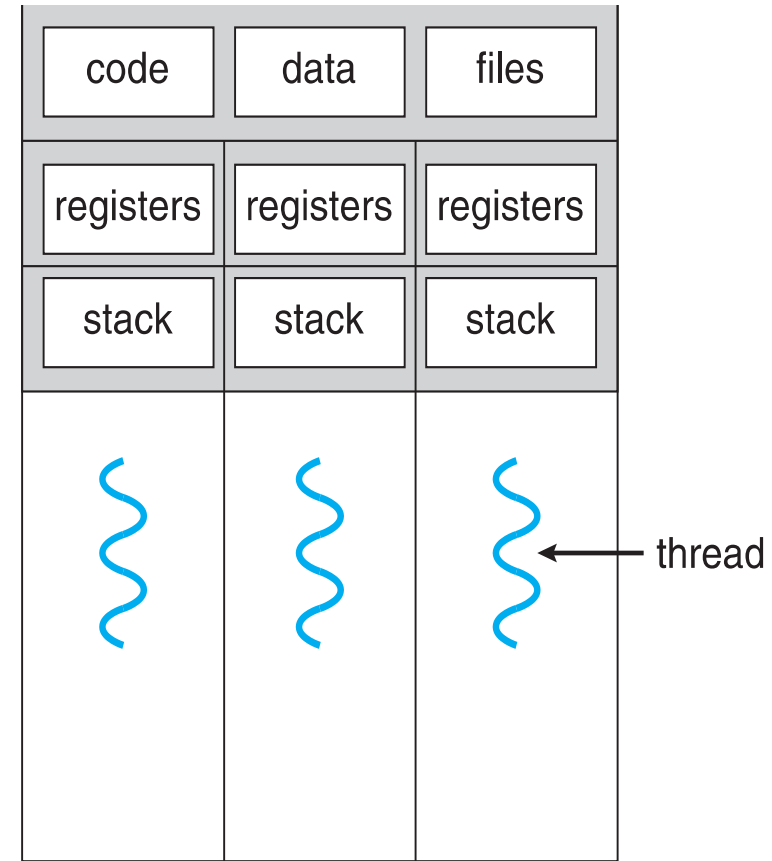
Introduction to Threads

- A thread is the **smallest unit of code** that can be executed by CPU.
- A process can be divided into multiple threads. The idea is to achieve parallelism by dividing a process into multiple threads.
- A thread is also known as **lightweight process**.
- Threads increase the utilization of multi-processor systems.
- **Increase the Responsiveness** of the task. Such as loading of Web pages.

Single and Multithreaded Processes



single-threaded process



multithreaded process

Processes Vs Threads

Process

- More resources
- Need context switch
- More context switch time
- In multiple processes each process operates independently of the others

Thread

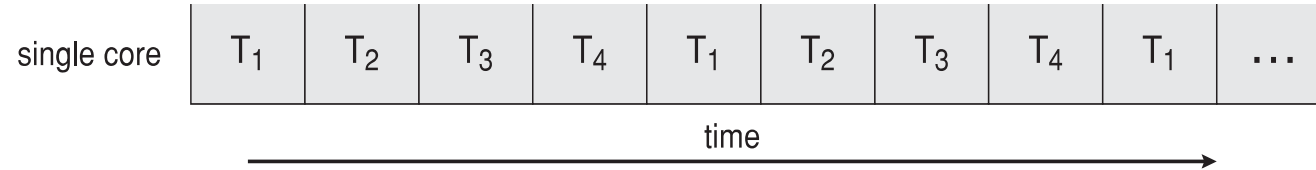
- Fewer resources
- User level threads don't need context switching
- Low context switching time for kernel level threads
- One thread can read, write or change another thread's data.

Benefits

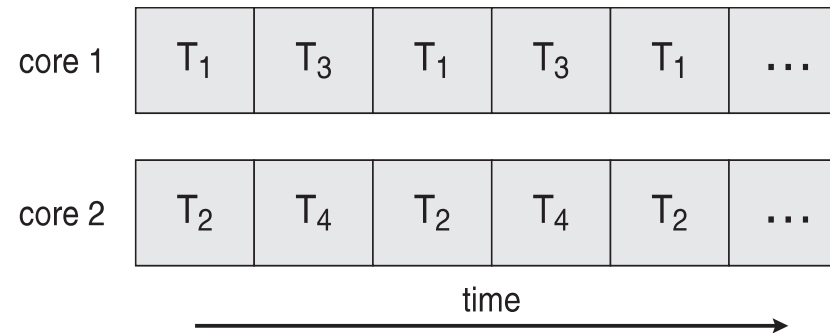
- **Responsiveness** – may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing
- **Economy** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – process can take advantage of multiprocessor architectures

Concurrency vs. Parallelism

Concurrent execution on single-core system:



Parallelism on a multi-core system:



Types of Threads

User Level Thread: User managed threads

Kernel Level Thread: Operating system manages threads acting on kernel (heart of OS)

User-Level Thread vs Kernel-Level Thread

User level thread

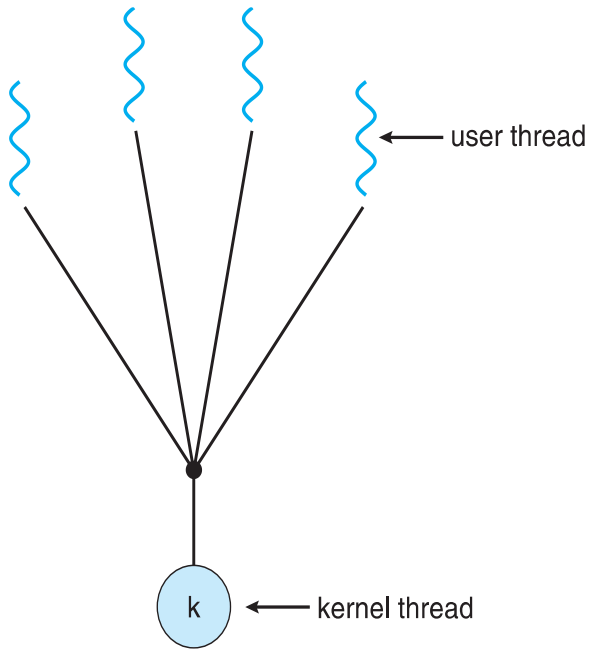
- Implemented by users
- The operating System doesn't recognize user-level threads.
- Implementation of User threads is easy.
- Context switch time is less.
- Context switch requires no hardware support.
- If one user-level thread performs a blocking operation then the entire process will be blocked.

Kernel level Thread

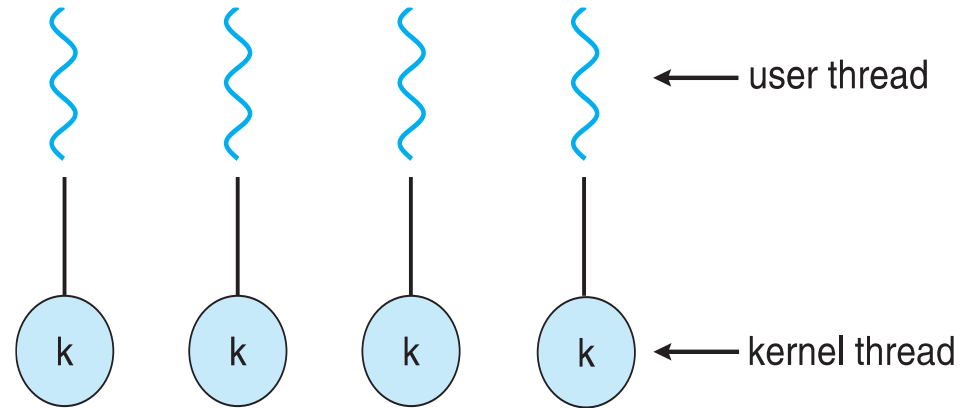
- Implemented by OS
- Kernel threads are recognized by Operating System.
- Implementation of Kernel-Level thread is complicated.
- Context switch time is more.
- Hardware support is needed.
- If one kernel thread performs a blocking operation then another thread can continue execution.

Multithreading Models

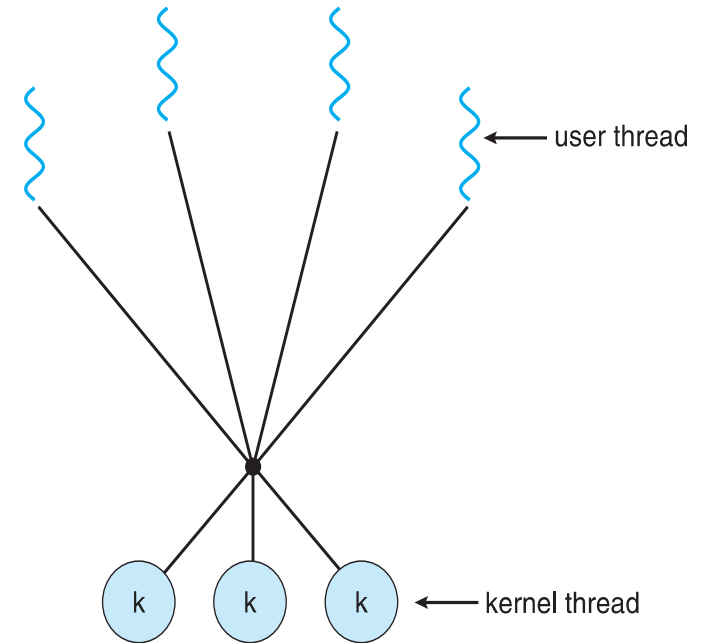
Many-to-One



One-to-One



Many-to-Many



Reference: Chapter 4